For now, take a look at Table 2-1.

Table 2-1. *Character ordinal values*

Character	Hex	Binary
H	0x48	01001000
E	0x45	01000101
L	0x4C	01001100
L	0x4C	01001100
O	0x4F	01001111
<SPACE>	0x20	00100000
W	0x57	01010111
O	0x4F	01001111
R	0x52	01010010
L	0x4C	01001100
D	0x44	01000100

A few people might be able to read the raw binary data and convert it to text in their head (and those people exist; I've met a few over the years), but the rest of us need software to do the job. If your program reads those bytes as text (e.g., when outputting via a `print` statement), then you should see "HELLO WORLD" as the output. Applications designed for the display of text data handle the interpretation for us, and all we need to worry about is the actual reading and understanding part.

Likewise, programs that perform mathematics will usually treat their data as numerical. If we take the byte values from our example above and interpret the first four as an integer (a whole number), then we get the numerical value of 1280066888, or 1212501068, depending on which end of the byte sequence the CPU starts at (this is known as "big-endian" versus "little-endian"). Modern computers generally use at least four bytes to store an integer.

```
Binary:
01001000 01000101 01001100 01001100
```